

2.1 RNA-seq - Expression Matrix

徐英泽 2024011267

[返回引导页](#)

1. 请阐述 RNA-seq 中归一化基因表达值的几种基本计算方法。

- CPM/RPM: Counts Per Million / Reads Per Million, 对测序深度的差异进行标准化, 不能比较不同长度的基因, 适合基因长度差异不大的 RNA, 如小RNA。
- RPKM: Reads Per Kilobase of transcript per Million mapped reads, 用于单端测序, 对长度与深度均进行标准化。
- FPKM: Fragments Per Kilobase of transcript per Million mapped fragments, 用于双端测序, 对长度与深度均进行标准化。
- TPM: Transcripts Per Million, 在FPKM的基础上, 将每个基因的FPKM值除以该样本中全部基因的FPKM值之和, 相当于在对基因长度校正后, 对文库大小再进行一次校正, 避免了不同文库间表达基因长度分布不一致造成的偏差。
- TMM Counts: Trimmed Mean of M-values, 用于edgeR, 消除样本间由于 RNA 组成差异带来的系统性偏移, 同时不假设绝大多数基因表达不变。
- RLE: Relative Log Expression, 用于DESeq2, 除样本间由于测序深度和 RNA 组成差异带来的系统性偏差。

2. 根据下述图片描述, 填出对应选项。

E D A

3. 通过软件计算, 判断给出文件shape02数据是来自哪一种sequencing protocols (strand nonspecific, strand specific - forward, strand specific - reverse), 并选择合适的参数计算shape02的read count matrix, 给出AT1G09530基因(PIF3基因)上的counts数目。

```
root@featurecount_docker:/home/test# /usr/local/bin/infer_experiment.py -r
GTF/Arabidopsis_thaliana.TAIR10.34.bed -i bam/Shape02.bam
Reading reference gene model GTF/Arabidopsis_thaliana.TAIR10.34.bed ... Done
Loading SAM/BAM file ... Total 200000 usable reads were sampled
```

```
This is PairEnd Data
Fraction of reads failed to determine: 0.0315
Fraction of reads explained by "1++,1--,2+-,2-+": 0.4769
Fraction of reads explained by "1+-,1-+,2++,2--": 0.4916
```

两种reads的比例接近, 认为是非链特异性文库。

```
root@featurecount_docker:/home/test# /home/software/subread-2.0.3-source/bin/featureCounts \
> -s 0 -p -t exon -g gene_id \
```

```
> -a GTF/Arabidopsis_thaliana.TAIR10.34.gtf \
> -o result/Shape02.featurecounts.exon.txt bam/Shape02.bam
# 进行featurecount。

root@featurecount_docker:/home/test/result# head Shape02.featurecounts.exon.txt
# Program:featureCounts v2.0.3; Command:"/home/software/subread-2.0.3-
source/bin/featureCounts" "-s" "0" "-p" "-t" "exon" "-g" "gene_id" "-a"
"GTF/Arabidopsis_thaliana.TAIR10.34.gtf" "-o" "result/Shape02.featurecounts.new.exon.txt"
"bam/Shape02.bam"
Geneid Chr      Start  End      Strand Length bam/Shape02.bam
AT1G01010      1;1;1;1;1 3631;3996;4486;4706;5174;5439
3913;4276;4605;5095;5326;5899 +;+;+;+;+ 1688 20
# 简要查看文件结构。

root@featurecount_docker:/home/test# awk -F'\t' 'NR>1 && $1=="AT1G09530" {print $7}'
result/Shape02.featurecounts.exon.txt
86
# 给出AT1G09530基因(PIF3基因)上的counts数目:86
```

4. tumor-transcriptome-demo.tar.gz提供了结肠癌(COAD), 直肠癌(READ)和食道癌(ESCA)三种癌症各50个样本的bam文件用featureCount计算产生的结果。请大家编写脚本将这些文件中的counts合并到一个矩阵中(行为基因, 列为样本), 计算logCPM的Z-score, 并用 heatmap 展示, 提供代码和heatmap。根据heatmap可视化的结果, 你认为这三种癌症中哪两种癌症的转录组是最相似的?

```
> setwd("C:/Users/54727/Downloads/tumor-transcriptome-demo")
>
> # ===== 1. 获取每个癌症文件夹下的文件名 =====
> COAD_names <- list.files(path = './COAD/')
> ESCA_names <- list.files(path = './ESCA/')
> READ_names <- list.files(path = './READ/')

> # ===== 2. 将这些文件中的counts合并到一个矩阵中(行为基因, 列为样本) =====
> COAD_tot <- read.table(paste('./COAD/', COAD_names[1], sep=''),
+                         header = T)[,c(1,7)]
> for (i in COAD_names){
+   temp <- read.table(paste('./COAD/', i, sep = ''),
+                       header = T)[,c(1,7)]      # 读取当前文件的1列与7列。
+   COAD_tot <- merge(temp, COAD_tot, all = T)    # 将当前数据与原有数据合并。
+ }
> row.names(COAD_tot) <- COAD_tot[,1]             # 以第一列作为行名。
> COAD_tot <- COAD_tot[,-1]                       # 将第一列删去(已经为行名)。
# 现在只剩纯矩阵了。

>
> ESCA_tot <- read.table(paste('./ESCA/', ESCA_names[1], sep=''),
+                         header = T)[,c(1,7)]
> for (i in ESCA_names){
+   temp <- read.table(paste('./ESCA/', i, sep = ''),
+                       header = T)[,c(1,7)]
```

```

+     ESCA_tot <- merge(temp, ESCA_tot, all = T)
+ }
> row.names(ESCA_tot) <- ESCA_tot[,1]
> ESCA_tot <- ESCA_tot[,-1]
>
> READ_tot <- read.table(paste('./READ/', READ_names[1], sep=''),
+                         header = T)[,c(1,7)]
> for (i in READ_names){
+     temp <- read.table(paste('./READ/', i, sep = ''),
+                         header = T)[,c(1,7)]
+     READ_tot <- merge(temp, READ_tot, all = T)
+ }
> row.names(READ_tot) <- READ_tot[,1]
> READ_tot <- READ_tot[,-1]
>
> # ===== 2. 计算logCPM的Z-score =====
> COAD_CPM <- t(1000000*t(COAD_tot)/colSums(COAD_tot))
> COAD_log <- log10(COAD_CPM+1) # 1为避免count为0时对数未定义。
> COAD_zscores <- (COAD_log - rowMeans(COAD_log))/apply(COAD_log,1,sd)
> COAD_zscores <- na.omit(COAD_zscores) # 删除NaN行。
>
> ESCA_CPM <- t(1000000*t(ESCA_tot)/colSums(ESCA_tot))
> ESCA_log <- log10(ESCA_CPM+1) # 1为避免count为0时对数未定义。
> ESCA_zscores <- (ESCA_log - rowMeans(ESCA_log))/apply(ESCA_log,1,sd)
> ESCA_zscores <- na.omit(ESCA_zscores) # 删除NaN行。
>
> READ_CPM <- t(1000000*t(READ_tot)/colSums(READ_tot))
> READ_log <- log10(READ_CPM+1) # 1为避免count为0时对数未定义。
> READ_zscores <- (READ_log - rowMeans(READ_log))/apply(READ_log,1,sd)
> READ_zscores <- na.omit(READ_zscores) # 删除NaN行。
>
> # ===== 3. 根据Z-score筛选数据 =====
> COAD_zscores <- pmin(pmax(COAD_zscores, -2), 2)
> ESCA_zscores <- pmin(pmax(ESCA_zscores, -2), 2)
> READ_zscores <- pmin(pmax(READ_zscores, -2), 2)

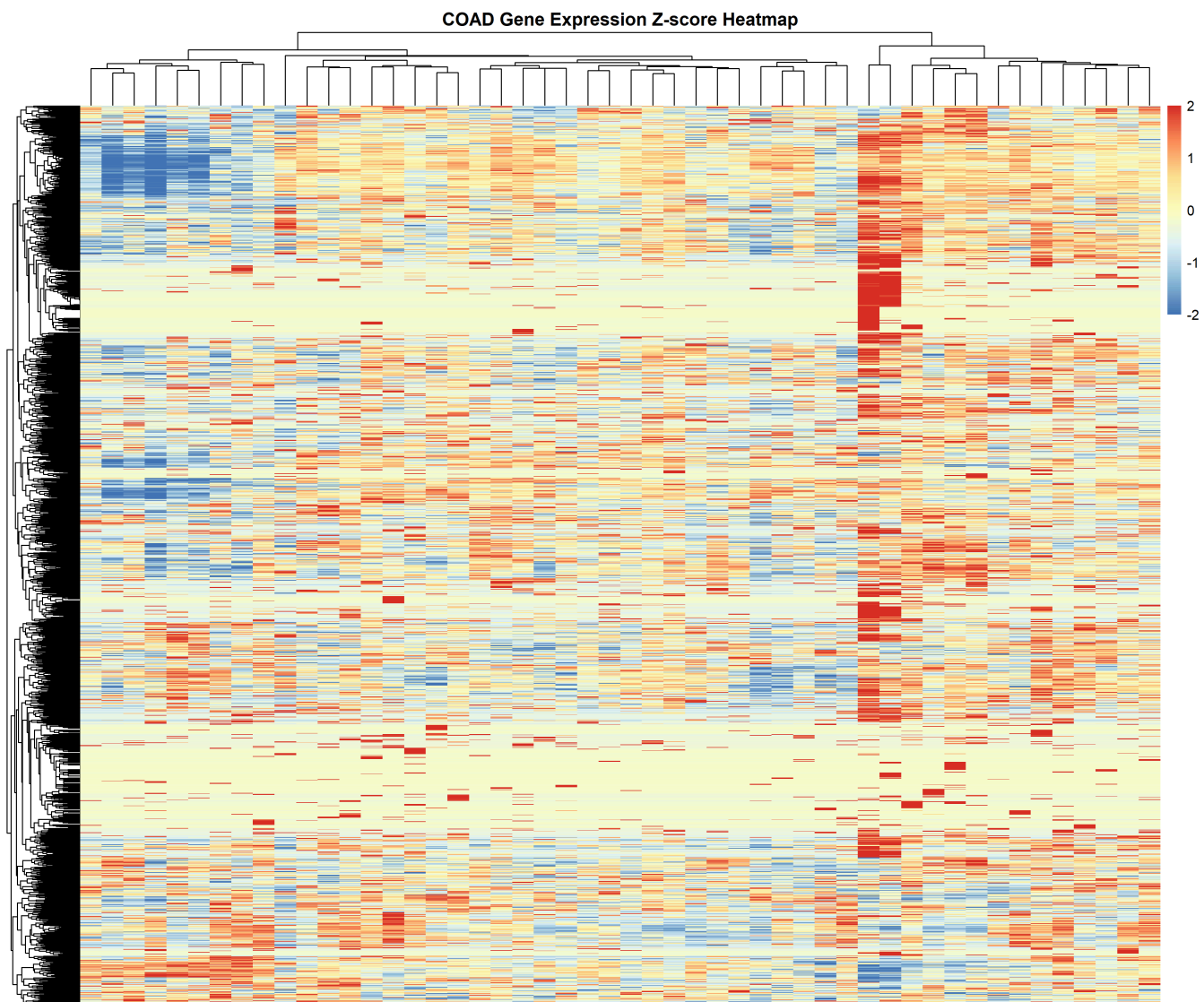
> # ===== 4. 绘制热图 =====
> library(pheatmap)
> pheatmap(as.matrix(COAD_zscores),
+          cluster_rows = TRUE,
+          cluster_cols = TRUE,
+          show_rownames = FALSE,
+          show_colnames = FALSE,
+          main = "COAD Gene Expression Z-score Heatmap",
+          filename = "C:/Users/54727/Desktop/COAD_heatmap.png",
+          width = 12,
+          height = 10)
> pheatmap(as.matrix(ESCA_zscores),
+          cluster_rows = TRUE,
+          cluster_cols = TRUE,
+          show_rownames = FALSE,
+          show_colnames = FALSE,
+          main = "ESCA Gene Expression Z-score Heatmap",

```

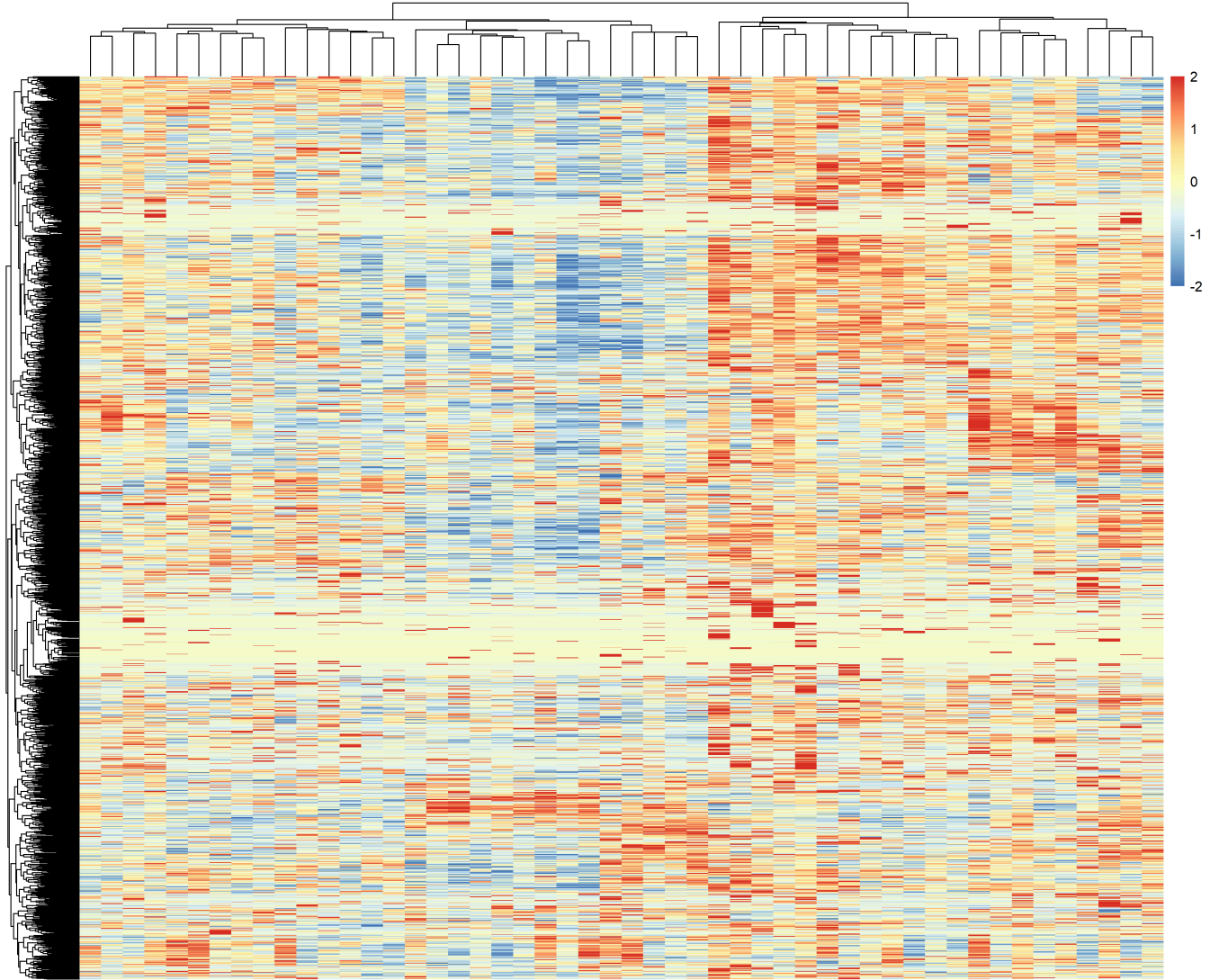
```

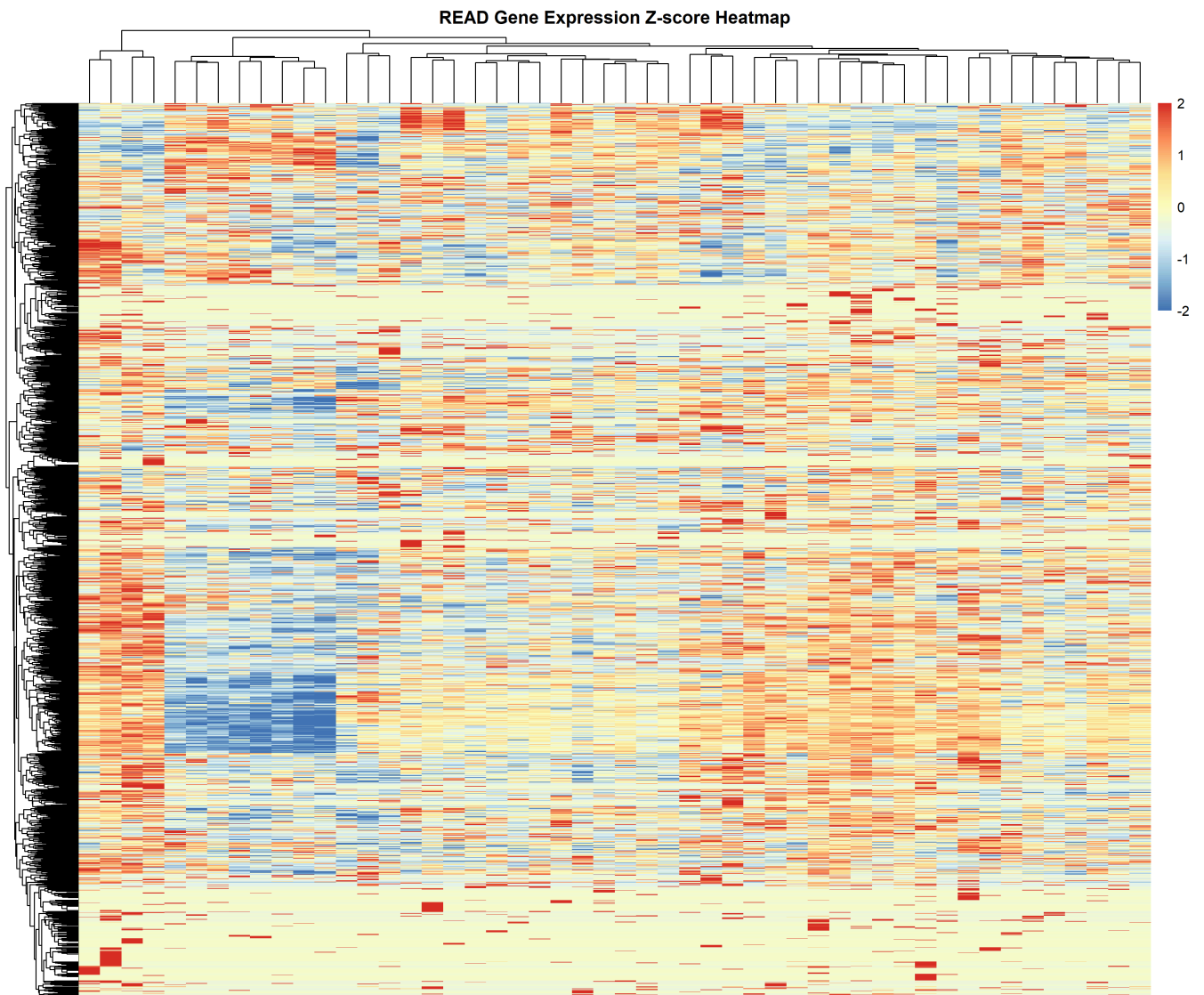
+     filename = "C:/Users/54727/Desktop/ESCA_heatmap.png",
+     width = 12,
+     height = 10)
> pheatmap(as.matrix(READ_zscores),
+     cluster_rows = TRUE,
+     cluster_cols = TRUE,
+     show_rownames = FALSE,
+     show_colnames = FALSE,
+     main = "READ Gene Expression Z-score Heatmap",
+     filename = "C:/Users/54727/Desktop/READ_heatmap.png",
+     width = 12,
+     height = 10)

```



ESCA Gene Expression Z-score Heatmap





热图结果如上图所示，COAD与READ比较相似。